Reviewer Pack — Integral Geometry Volume Bounds Discrepancy in Greater-Than …

Entry 2cc8c1f1ceea · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 21:44:46 UTC

Integral Geometry Volume Bounds Discrepancy in Greater-Than Communication Complexity

Entry ID: `2cc8c1f1ceea` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-20 21:44:46 UTC

1. Conjecture

Field A (mathematical branch): Integral Geometry **Field B** (complexity object): Communication Complexity of Greater-Than Problem

Statement:

> For a random 3-CNF instance with n variables, the discrepancy of the Greater-Than communication problem is Θ(∫_{S^{n-1}} vol(Π_i (x_i - y_i ≥ 0)) dσ), where Π_i denotes the product of halfspaces and σ is the uniform measure on the unit sphere.

Rationale (proposer's reasoning):

> Integral geometry's measure-theoretic tools can quantify the 'spread' of geometric regions defined by input constraints, potentially exposing hidden structure in communication complexity that standard discrepancy bounds miss.

Taxonomy category: `DISPERSION_DISCREPANCY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `735bdaaf7e15dd82`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(10 * n):  # Generate enough clauses to cover all variables
            clause = set()
            for _ in range(3):
                var = random.randint(1, n)
                if random.choice([True, False]):
                    clause.add(var)
                else:
                    clause.add(-var)
            clauses.append(clause)
        return clauses

    def discrepancy(clauses):
        n = len(clauses[0])
        disc = 0
        for x in range(2**n):
            if all(x & (1 << i) == y & (1 << i) or (x & (1 << i)) * (y & (1 << i)) >= 0 for clause
                disc += 1
        return disc / 2**n

    def spherical_integral(n):
        # This is a placeholder function. Actual implementation would involve discretizing the uni
        return 1.0  # Placeholder value

    n = 40
    clauses = generate_3cnf(n)
    disc = discrepancy(clauses)
    integral = spherical_integral(n)

    return {
        "metric_name": "discrepancy_communication_complexity",
        "metric_value": disc,
```

```
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_disc = sum(r["metric_value"] for r in results) / len(results)
    std_disc = math.sqrt(sum((r["metric_value"] - mean_disc)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_disc} std={std_disc} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73d0f737.py", line 65, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73d0f737.py", line 48, in run_trial
    disc = discrepancy(clauses)
           ^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73d0f737.py", line 38, in discrepancy
    if all(x & (1 << i) == y & (1 << i) or (x & (1 << i)) * (y & (1 << i)) >= 0 for clause, y in zip(clauses,
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73d0f737.py", line 38, in <genexpr>
    if all(x & (1 << i) == y & (1 << i) or (x & (1 << i)) * (y & (1 << i)) >= 0 for clause, y in zip(clauses,
                                  ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to undefined variable 'i' in the discrepancy calculation, preventing data collection. | next: Fix the code's variable indexing logic and re-run tests with corrected clause-variable mapping

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	82534
2	preregistration	ollama_remote	qwen3:8b	0	0	31668
3	novelty	ollama_remote	qwen3:8b	0	0	27403
4	novelty	ollama_remote	qwen3:8b	0	0	17387
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10402
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6751
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9703
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7460
9	judge	ollama_remote	qwen3:8b	0	0	27966

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 221274 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/2cc8c1f1ceea.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2cc8c1f1ceea.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2cc8c1f1ceea.tar.gz` (if generated)